

Automation & QA Developer

Take-Home Skills Assessment

Role	Automation & QA Developer
Primary focus	Keep AI-built (“vibe-coded”) web apps running smoothly and build/maintain automation workflows in n8n.
Core skills tested	Web application debugging & QA, API integration, n8n workflow design, error handling, written communication.
Time limit	Spend no more than 2 hours total. We value clarity and reasoning over polish.
How to submit	In the assignment submission section here

1. Before You Begin

Read the entire assignment first. You may use any documentation, AI assistants, or Google — but be prepared to explain every decision in a follow-up interview. If you get stuck, write down your assumptions and move on; we care about how you think, not just whether you finish.

If you don’t already have an n8n instance, install the free self-hosted version locally (Docker or npx) or use the free n8n Cloud trial.

2. Task 1 — Web App QA & Debug Report (~75 min)

Goal: prove you can take a half-working web app, find what’s wrong, and explain it clearly to a non-technical stakeholder.

Scenario

You’ve just been handed a small web app that was built by a non-developer using AI-assisted (“vibe coding”) tools. Users have reported “it feels broken” but no one has documented exactly what’s wrong. Your job is to investigate and report.

What to do

- Pick any one of the following demo apps (or, if we sent you a private repo link, use that instead):
 - <https://github.com/gothinkster/realworld> — pick any frontend + backend implementation.
 - <https://demo.realworld.io> — public live demo if you don’t want to run anything locally.
 - Any small open-source SaaS-style app you can run in under 15 minutes.
- Run the app locally (or use the live demo) and exercise the main user flows: sign-up, login, create content, edit, delete, log out.

3. Find at least 5 issues. These can be functional bugs, UX problems, broken validation, missing error handling, slow responses, security smells, accessibility issues, or anything else that would block production use.
4. For each issue, document it using the template in section 5 (one row per issue).
5. Pick ONE of the issues and write a short root-cause analysis (5–10 sentences): what is happening, why, and how you would fix it.

Deliverable

- A single PDF or Word document titled “Task1_QA_Report_[YourName].pdf” containing the bug table and the root-cause analysis.
 - Screenshots or short screen-recordings are encouraged but not required.
-

3. Task 2 — n8n API Integration Workflow (~75 min)

Goal: prove you can wire third-party APIs together in n8n with clean error handling.

Scenario

We need a workflow that watches a public data source, enriches it with a second API call, and posts a digest to a notification channel. Imagine this is the daily “morning brief” our team will rely on.

Requirements

- Trigger: either a Schedule trigger (every 1 hour, for testing) OR a Webhook trigger you can hit with curl/Postman.
- First HTTP Request: call any free public API of your choice. Suggested options: GitHub REST API (e.g. trending repos for a topic), OpenWeather, JSONPlaceholder, CoinGecko, NewsAPI, or HackerNews.
- Transformation: use a Function/Code node OR n8n expressions to reshape the response — filter, sort, or keep only the top 5 results.
- Second HTTP Request: enrich at least one of those items by calling a second API (or a second endpoint of the same API). Example: take the top GitHub repo and fetch its README, or take the top crypto and fetch its 24h chart.
- Conditional branch: IF node that routes results differently based on a threshold you choose (e.g. “if stars > 1000”).
- Output: send the final digest to one of — your own email, a Discord webhook, a Slack webhook, a Telegram bot, or a Google Sheet. (Free options all work.)
- Error handling: the workflow must NOT crash silently. Use the “Continue On Fail” / Error Trigger / On Error branch so a failed API call is logged or routed to a fallback.
- Credentials: do NOT hard-code secrets. Use n8n’s Credentials store and reference them from the nodes.

Deliverables

- Exported workflow JSON file: “Task2_Workflow_[YourName].json” (use n8n’s built-in Download menu).
- Two screenshots: (a) the workflow canvas, (b) one successful execution showing the output data.
- A short README (max 1 page) explaining: which APIs you used, why, what your transformation does, and how the error path behaves.

4. Bonus Task — Uptime Monitor (Optional, ~30 min)

Goal: prove you can combine the two skills above. Strong submissions almost always attempt this.

Build a second n8n workflow that pings the web app you tested in Task 1 every 5 minutes, checks the HTTP status code, and posts an alert to the same notification channel from Task 2 if the app responds with anything other than 200. Bonus points for retry logic, response-time tracking, and a daily summary.

Deliverable

- Exported workflow JSON: “Bonus_UptimeMonitor_[YourName].json” and one screenshot of the workflow.

5. Bug Report Template (for Task 1)

Copy this table into your Task 1 report and fill one row per issue. Severity should be one of: Critical, High, Medium, Low.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1					
2					
3					
4					
5					

7. Submission Checklist

Share the Github link of the work you have done, with the documentation.
Also share Loom video, or anything, explaining your work, logic etc.

8. Tips & Frequently Asked Questions

Can I use AI tools like ChatGPT or Claude?

Yes. We use them ourselves. Just be prepared to defend every choice in a 30-minute follow-up call — including “why this API, why this node, why this threshold.”

My workflow works but it’s ugly. Should I prettify it?

No. Working > pretty. Spend leftover time on error handling or the bonus task instead.

What if the demo app I picked has no obvious bugs?

Pick a different one, or add load (e.g. very long inputs, special characters, network throttling, expired session) to surface issues. Real production bugs hide in edge cases.

Do I need to write tests?

Not required. If you do, a single happy-path test against your n8n webhook (e.g. with curl) is plenty.

How will you use my submission?

Only for evaluating your application. We won't deploy your workflows or reuse your code. Strong submissions move to a 45-minute technical conversation.

What to Submit?

- The Github link of your work (Please make it Public).
- A brief Documentation of work you have done (less than 1 page)
- Loom Video explaining whatever you have done

Good luck — we're looking forward to seeing how you think.